

Reviewer Pack — Minimal Rank of Tropical Vector Spaces Bounds Communication ...

Entry 0838455fd40f · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-29 22:16:04 UTC

Minimal Rank of Tropical Vector Spaces Bounds Communication Complexity for XOR

Entry ID: 0838455fd40f **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-29 22:16:04 UTC

1. Conjecture

Field A (mathematical branch): Tropical Geometry **Field B** (complexity object): Boolean Function Complexity: Circuit Complexity of XOR

Statement:

For any XOR boolean function f with n variables, the minimal rank r_f of the associated tropical vector space T_f is at most $O(\log(n))$ and there exists a constant c such that for all k -sat instances F with m clauses on n variables, $D_2(F) \leq c \log^2(m)$, where $D_2(F)$ is the Tusnady 2-box discrepancy and T_F is the tropical vector space associated with the boolean function representing F .

Rationale (proposer's reasoning):

Tropical geometry has emerged as a powerful tool in understanding complexity classes by providing geometric interpretations of computational problems. The minimal rank of a tropical vector space may capture essential properties of the underlying boolean functions that affect their circuit complexity. By linking this invariant to communication complexity, we could potentially find new ways to understand and prove lower bounds for XOR circuits.

Taxonomy category: cg_kw_andreev (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 59e3d4477580e683

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the Spearman rank correlation coefficient (r) between the minimal rank of tropical vector spaces and Tusnady 2-box discrepancies across 30 random seeds is ≥ 0.5 , indicating a significant positive relationship.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	1.00	SAFE	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	SAFE	UNCERTAIN
KARP_LIPTON	SAFE	1.00	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "tropical geometry" AND "communication complexity" AND XOR" - "minimal rank tropical vector spaces" AND Boolean function complexity" - "Tusnady 2-box discrepancy" AND $c \log^2(m)$ AND associated tropical vector space"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=124, elapsed=244.0s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_xor_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def compute_tropical_vector_space(f):
        n = int(math.log2(len(f)))
        T_f = [[float('inf')] * (2**n) for _ in range(2**n)]
        for i in range(2**n):
            for j in range(2**n):
                if 2**i < len(f) and 2**j < len(f):
                    T_f[i][j] = min(f[2**i], f[2**j])
        return T_f

    def tusnady_2_box_discrepancy(T_f, n):
        m = len(T_f)
        D_2 = 0
        for i in range(m):
            row_max = max(T_f[i])
            col_max = max(row[T_i] for row in T_f for T_i in range(m) if (T_i & i) == i)
            D_2 += abs(row_max - col_max)
        return D_2 / m

    def spearman_rank_correlation(ranks):
        n = len(ranks)
```

```

sorted_ranks = sorted(range(n), key=lambda x: ranks[x])
rank_order = [sorted_ranks.index(i) for i in range(n)]
n_ranks = list(range(1, n + 1))
sum_diff_squares = sum((n_ranks[i] - rank_order[i]) ** 2 for i in range(n))
return 1 - (6 * sum_diff_squares) / (n * (n**2 - 1))

n_values = [5, 10, 15, 20, 30, 40]
ranks_r_f = []
ranks_D_2 = []

for n in n_values:
    f = generate_xor_function(n)
    T_f = compute_tropical_vector_space(f)
    D_2 = tusnady_2_box_discrepancy(T_f, n)

    r_f = sum(f) / len(f)
    ranks_r_f.append(r_f)
    ranks_D_2.append(D_2)

correlation_coefficient = spearman_rank_correlation(ranks_r_f, ranks_D_2)

return {
    "metric_name": "Spearman rank correlation coefficient",
    "metric_value": correlation_coefficient,
    "instances_tested": len(n_values),
    "n_max": max(n_values),
    "conjecture_holds": correlation_coefficient >= 0.5,
    "counterexample": "" if correlation_coefficient >= 0.5 else "Spearman rank correlation coefficient < 0.5"
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 1 for i in range(30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"Spearman rank correlation coefficient < 0.5\" for seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

TIMEOUT after 240s

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test timed out before producing data, which means that the Spearman rank correlation coefficient could not be calculated to determine if the conjecture is supported or falsified. | next: Run the test again with a longer timeout and ensure it completes successfully.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	29754
2	preregistration	ollama_remote	glm4:latest	0	0	18725
3	novelty	ollama_remote	glm4:latest	0	0	13513
4	novelty	ollama_remote	glm4:latest	0	0	12559
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16382
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	47821
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	24026
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11633
9	judge	ollama_remote	glm4:latest	0	0	42598

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 217011 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/0838455fd40f.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/0838455fd40f.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/0838455fd40f.tar.gz` (if generated)